

AFTER ACTION REPORT

LL-2026-04-23-001

ai_bridge Lua addon rewrite: Ashita v4 to v3 classic API

Classification: SIGNIFICANT

Status: PUBLISHED

Date: 2026-04-23

Owner: Chharith Oeun | Reporter: Chharbot (Claude Opus 4.7)

LIVING DOCUMENT

CUI - CONTROLLED UNCLASSIFIED INFORMATION

Executive Summary - Top 3 Lessons

1. Ashita v3 classic uses `_addon.name + ashita.register_event`; v4 uses `addon.name + ashita.events.register`. Silent stalls at load stage 2/3 when mismatched.
2. When an addon has NO console/chat output, a file-based `io.open()` trace log at each step is the fastest diagnostic path.
3. Before coding any new Ashita Lua addon, dump a known-working addon (`distance.lua`, `tparty.lua`) first to confirm which API flavor the runtime expects.

1. Situation

Task #37 required the `ai_bridge` Ashita Lua addon to bind `127.0.0.1:27115` and respond to JSON-RPC ping so `chharbot smoke-test.ps1` reports '`ai_bridge`: up'. An earlier version of `ai_bridge.lua` (16,944 bytes, ~447 lines) was staged and loaded cleanly at Ashita startup but the addon stalled before opening the port. The Ashita debug log (`F:\ffxi\Ashita\logs\04_22_2026 00.17.15.txt`) showed `ai_bridge` reaching 'Addon load. (2) - `ai_bridge`' but never '(3)', while every other addon (`distance`, `fps`, `timestamp`, `tparty`) reached stage (3) within ms. Smoke test therefore continued to report `ai_bridge` as down, blocking closure of Task #37.

2. Timeline

Time	Event
2026-04-22 00:17	Ashita + FFXI launched clean. <code>pol.exe</code> pid 31756 connected to LSB zones. <code>Default.txt</code> ran <code>/addon load ai_bridge</code> .
2026-04-22 00:17:21	Ashita log shows <code>ai_bridge</code> reaches stage (2), never stage (3). Port 27115 stays DOWN.
2026-04-22 00:28	<code>ASHITA_FULL_LOG.ps1</code> : grepped Ashita log for <code>ai_bridge/addon/socket/bind/error</code> — confirmed: silent stall, no <code>[E]</code> error line, no bind error, no Lua exception printed.
2026-04-22 00:29	<code>AI_BRIDGE_DUMP.ps1</code> : discovered <code>addons/libs/chat.lua</code> MISSING; <code>json.lua</code> at NESTED <code>libs/json/json.lua</code> path (not flat <code>libs/json.lua</code>).
2026-04-22 00:30	<code>LIBS_DUMP.ps1</code> : dumped <code>distance.lua</code> + <code>tparty.lua</code> sources — discovered they use <code>_addon.name / ashita.register_event</code> (Ashita v3 CLASSIC API), not the v4 <code>addon.name / ashita.events.register</code> API <code>ai_bridge</code> was written against.
2026-04-22 00:33	Rewrote <code>ai_bridge.lua</code> for v3 classic API (11,684 bytes): <code>_addon.*</code> , <code>ashita.register_event</code> , <code>require('json.json')</code> , <code>AshitaCore:GetDataManager</code> , file trace log. Staged as <code>ai_bridge_v3.lua</code> .
2026-04-22 00:34	<code>AB_DEPLOY_V3.ps1</code> : backed up broken v4 as <code>ai_bridge.lua.v4-backup</code> ; deployed v3 version to <code>F:\ffxi\Ashita\addons\ai_bridge\ai_bridge.lua</code> ; killed <code>pol.exe</code> 31756 to force reload.
2026-04-22 00:35	<code>AB_RELAUNCH.ps1</code> : launched Ashita 32068 non-elevated (RUNASINVOKER). Manual click Launch Game (841,280) → <code>pol.exe</code> 5772 spawned.
2026-04-22 00:36	<code>AB_STATUS.ps1</code> : trace log shows 'load event fired → <code>socket.bind => tcp{server} err=nil</code> → listening on <code>127.0.0.1:27115</code> '. Ashita log: <code>ai_bridge</code> reaches stage (3). Port 27115 = UP.
2026-04-22 00:38	<code>AB_PING.ps1</code> : TCP connect to <code>127.0.0.1:27115</code> , sent <code>{jsonrpc:2.0,id:1,method:ping}</code> , received <code>{id:1,jsonrpc:2.0,result:{ok:true,ts:1776843519}}</code> .
2026-04-22 00:38	<code>SMOKE_RUN3.bat</code> : <code>chharbot smoke-test</code> reports ' <code>ai_bridge</code> : up', ' <code>lsb_admin_api</code> : up', ' <code>3 ok / 0 empty</code> ', exit code 0. Task #37 closed.
2026-04-23 17:31	<code>AB_COMMIT.bat</code> : staged <code>addons/ai_bridge/ai_bridge.lua</code> only, committed 007ec4b 'fix(ai_bridge): rewrite addon for Ashita v3 classic API (Task #37)', pushed to origin/main.

3. Root Cause Analysis

PRIMARY: ai_bridge.lua was coded against the Ashita v4 Lua API (addon.name, ashita.events.register, chat.header/error, T{} table wrapper), but the deployed Ashita runtime at F:\ffxi\Ashita is the original Ashita v3 CLASSIC (atom0s 2014-2017 branch) which uses a different API surface: _addon.name globals, ashita.register_event (no .events.), no chat module, plain Lua tables.

WHY SILENT: top-level require() calls succeeded (Ashita v3 classic happens to also expose an ashita global), so stage (2) Lua-chunk-executed completed cleanly. The load event registration line `ashita.events.register('load', 'ai\_bridge\_load', fn)` evaluated to nil-index on the missing events subtable, but Ashita caught the error inside the addon boot harness without surfacing any [E] line to the debug log. No chat print, no Lua traceback in console, no crashlog. The only symptom was the absence of the 'Addon load. (3) - ai_bridge' line.

CONTRIBUTING: (a) the original v4 ai_bridge.lua had no runtime version assertion (`assert(\_addon, 'expected Ashita v3 classic')`) that would have failed loudly; (b) before coding, no working addon was dumped from the deployed Ashita to confirm which API flavor the runtime expects; (c) the two API flavors use overlapping global names (`ashita`, `AshitaCore`) which made it easy to assume they were compatible.

4. Attempts That Failed

Attempt: Assume socket.bind was failing (port in use, addr bind error)

Reasoning: Initial hypothesis for 'listening never prints' is always bind failure.

Why failed: PORT_CHECK2 showed 27115 DOWN and no process holding it. trace log later confirmed bind succeeded cleanly — the handler never fired at all.

Learned: When stage (2) never advances to (3), the handler isn't running; don't waste time instrumenting the bind itself.

Attempt: Suspect require('json') resolving to nil due to nested libs/json/json.lua path

Reasoning: The addon wraps json load in pcall and tolerates nil, but maybe some downstream code blew up on missing json.

Why failed: json is only used in request/response encode/decode, not in load handler. The load handler would fire before any request arrived. Moreover the rewrite uses require('json.json') explicitly and works.

Learned: pcall-wrapped requires are fine to leave broken if the addon tolerates nil; don't chase them when stage (3) never reaches.

5. Solutions That Worked

Dump a working Ashita addon to learn the real API surface (source: internal-reference)

LIBS_DUMP.ps1 + V3_REF.ps1 printed full contents of distance.lua and tparty.lua. Both start with `\_addon.name = '...'`, then `require 'common'`, then `ashita.register\_event('load', function() ... end)`. This is the v3 classic pattern. No `addon.name`, no `ashita.events.register`, no `chat.header`.

Rewrite ai_bridge.lua for v3 classic API (source: derived)

Replaced `addon.\*` → `\_addon.\*`, `ashita.events.register(evt, id, fn)` → `ashita.register\_event(evt, fn)`, `chat.header/error` → plain `print(...)` and io.open-based trace file, `T{}` → plain Lua `{}`, `require('json')` → `require('json.json')`, `AshitaCore:GetMemoryManager():...` → `AshitaCore:GetDataManager():...`, `AshitaCore:GetChatManager():QueueCommand(txt, 1)` for send_text.

File-based trace log for silent addon failures (source: invented)

At the top of the addon and inside the load/unload handlers, `local f = io.open('F:\ffxi\Ashita\logs\ai\_bridge\_trace.log', 'a'); f:write(...); f:close()`. Works even when print() output is suppressed, chat manager is not yet available, or errors are eaten by Ashita's addon harness.

Non-blocking socket poll via render event (source: derived)

ashita.register_event('render', fn) fires every frame. Inside: accept() with setTimeout(0), then for each client pump receive(1024), split on newlines, dispatch JSON-RPC. Never blocks the game thread.

6. New Capabilities Unlocked

Capability	Description	Location	Status
Ashita v3 classic addon template	11,684-byte ai_bridge.lua with _addon/register_event/DataManager/render-poll/file-trace boilerplate	F:\ffxi\lsb-repo\addons\ai_bridge\ai_bridge.lua	committed 007ec4b
File-based Lua trace logger	Writes per-step progress to a file on disk, survives silent failures	embedded in ai_bridge.lua	PROVEN
API mapping table: Ashita v3 vs v4	_addon.name vs addon.name; register_event vs events.register; GetDataManager vs GetMemoryManager; print vs chat.header; {} vs T{}	this AAR + ai_bridge.lua	DOCUMENTED
RUNASINVOKER boot workaround for Ashita	HKCU AppCompat Layers bypass for manifest-declared elevation, new boot-v2 path for xiloader pol.exe	from prior session, still in use	PROVEN

7. Known Bugs and Workarounds

ID	Description	Workaround	Pri	Status
AB-001	cowork sandbox mount detaches silently when underlying drive drops; subsequent writes go to overlay, not the real drive, and reads still see old sandbox view	Re-call request_cowork_directory after drive reconnect; or bypass sandbox and write scripts directly via Notepad on Windows	P3	WORKAROUND
AB-002	Ashita v3 classic swallows errors in addon load handler with no log entry and no crashlog; only symptom is missing 'Addon load. (3)' line	Always ship addons with file-based trace log at each step; add "assert(_addon, 'Ashita v3 required')" runtime gate	P2	OPEN
AB-003	External USB enclosure F: drive disconnected during active session; dev environment fully offline until power-cycle	Commit and push often (not end-of-session); verify after reconnect that mount actually re-bridges	P3	OPEN

8. Warfighter Analysis

Foreseeable: YES. Ashita v3 classic and v4 fork have publicly documented API differences. A 15-second check of a working addon's source would have revealed the flavor.

Red Team: Assumed the Ashita install was modern/v4 without verification. Assumed silent stall meant bind failure rather than event-registration failure.

Blue Team: Ashita debug log told us exactly where execution stopped (stage 2 vs 3). Working addon sources (distance.lua, tparty.lua) were on disk and readable. File trace log pattern was invented on the spot and paid for itself immediately.

White Team: Debug session was efficient once the API mismatch was named. Total wall-clock from 'port still DOWN' to 'SMOKE : up' was ~20 minutes.

Vulnerability: WEAK - The failure mode was silent — no error, no crash, no alert. Only a missing log line. Without the file trace log invention, this could have burned hours.

9. Token Cost Estimate

Estimate: ~35K tokens for the v4-to-v3 debug + rewrite + verify cycle, plus ~8K for this AAR.

Waste: None identified — each diagnostic script directly advanced the investigation.

Savings: Dumping distance.lua / tparty.lua source BEFORE writing the original v4 ai_bridge.lua would have saved the entire debug cycle (~30K tokens). Folded into P1 recommendation.

10. Doctrine Update

BOOTSTRAP.md: Add 'Ashita addon pre-flight' checklist: before writing a new Lua addon, dump one working addon from the target install to confirm v3 classic vs v4 fork API.

addons/ai_bridge/README.md: Document the v3 classic API assumptions and link to distance.lua/tparty.lua as reference implementations.

MASTER-LESSONS.md: Append this report's 3 key lessons.

11. Recommendations and Next Actions

P1: Add runtime version assertion to ai_bridge.lua top: `assert(\_addon and ashita.register\_event, 'ai\_bridge requires Ashita v3 classic API')`. Fail loudly, not silently.

P1: Write a 1-page cheatsheet in docs/ashita-v3-vs-v4.md documenting the API mapping table. Reference it from the Ashita section of the project README.

P2: Create an Ashita v3 classic addon template at addons/_template/ with the minimum boilerplate (load/unload/render, trace log, render-poll socket) so future addons start from a known-good base.

P2: Add a pre-commit hook or CI check that greps Lua files for v4-only symbols (addon.name, ashita.events.register, chat.header) and rejects them with a helpful message.

P3: Investigate the cowork mount detachment bug (AB-001) — see if there's a refresh primitive that re-bridges without re-requesting the directory.

P3: Set up a scheduled git status check on F:\ffxi\lsb-repo that warns when working tree has been dirty for >4 hours, to catch 'forgot to commit before drive dropped' situations.